SYNOPSIS

on

HireSense AI

An AI-Based Resume Screening and Recruitment Assistant

Submitted in partial fulfilment of the requirements for the award of the degree of

[Bachelor of Technology in Computer Science and Engineering]

Submitted by

[Student Name] – [Enrollment / Roll No.]

[Student Name] – [Enrollment / Roll No.]

Under the guidance of

[Guide Name]

[Designation, Department]

[Department of Computer Science and Engineering]

[Name of College / University]

[City, State]

Academic Year 2026 – 2027

TABLE OF CONTENTS

Broad Area of Work	**3**
Introduction	**3**
Literature Survey / Work done in the field of proposed work	**4**
Existing Gaps	**6**
Objectives of the proposed work	**7**
Proposed Methodologies	**8**
Expected Outcome of the proposed work	**13**
Future scope of the work	**14**
References	**14**

1. Broad Area of Work

The proposed work, titled “HireSense AI – An AI-Based Resume Screening and Recruitment Assistant”, belongs to the broad area of Artificial Intelligence applied to Human Resource Technology (HR Tech), also called e-recruitment or talent acquisition technology. It brings together several sub-areas of computer science:

- **Natural Language Processing (NLP):** extracting names, contact details, education, work experience, projects, certifications and skills from unstructured resume text, and understanding the meaning of job descriptions.
- **Semantic matching and information retrieval:** comparing a resume with a job description by meaning and context rather than by exact keyword overlap.
- **Generative AI and Large Language Models (LLMs):** producing explanations, resume improvement suggestions, interview questions and conversational answers grounded in the stored data.
- **Responsible and explainable AI:** keeping scores explainable, validating extracted information against the source text (hallucination control), and reducing bias through blind screening and fairness audits.
- **Full-stack web engineering and databases:** a role-based web application built with Next.js and Node.js and backed by a MySQL relational database.

Keywords: Resume screening, Applicant Tracking System (ATS), Natural Language Processing, Large Language Model, Semantic Matching, Skill Gap Analysis, Explainable AI, Bias Detection, Next.js, MySQL.

2. Introduction

2.1 Background

Recruitment is one of the most time-consuming activities of any organisation. A single job opening, especially for entry-level and technical roles, can attract hundreds of applications, and each resume has to be read, compared with the job requirements and ranked. Manual screening is slow, tiring and inconsistent: two recruiters may rate the same resume differently, and a tired reader may overlook a strong candidate. Many organisations therefore use an Applicant Tracking System (ATS) that filters resumes automatically.

Traditional ATS products mostly depend on keyword matching. A resume that says “built REST services with Express” may be rejected for a job asking for “Node.js backend development” simply because the exact words differ. At the same time, job seekers are often unaware why their resumes are rejected and have little guidance on how to improve them. Recent advances in NLP and Large Language Models make it possible to understand the

meaning of text, to explain decisions in plain language and to structure unstructured documents automatically.

2.2 Problem Statement

To design and develop a web-based system that automatically extracts structured information from resumes, matches candidates with job descriptions using semantic understanding, ranks them with explainable scores, helps recruiters through the complete hiring workflow (shortlisting, comparison, interview scheduling, analytics) and helps candidates improve their resumes, while keeping the final hiring decision with a human and guarding against incorrect AI output and unfair bias.

2.3 Proposed System in Brief

HireSense AI is a role-based platform with three types of users:

- **Candidate:** registers, uploads several versions of a resume, receives an ATS compatibility report, improvement suggestions and a skill gap analysis, browses jobs recommended for the resume, applies, tracks the application status and uses a generative AI career assistant.
- **Recruiter:** creates and publishes job openings with required and optional skills, uploads many resumes at once for batch screening, views ranked candidates with an explainable matching report, compares candidates, shortlists or rejects them, schedules interviews and generates interview questions, and queries the candidate database through an HR chatbot.
- **Administrator:** manages users and job postings and monitors platform-wide statistics, skill demand, API usage and errors, and audit logs.

2.4 Motivation and Need

- Reduce the time and effort spent on first-round resume screening.
- Provide consistent, explainable evaluation instead of unexplained numeric scores.
- Give candidates actionable feedback rather than silent rejection.
- Demonstrate a practical, responsible use of LLMs where output is verified and humans stay in control.

3. Literature Survey / Work done in the field of proposed work

3.1 Evolution of Automated Resume Screening

Early automated screening relied on Boolean search and keyword counting. Term-weighting schemes such as TF-IDF [1] improved the ranking of documents by giving more importance to rare, informative words, and they are still the basis of many keyword-driven ATS products.

However, lexical methods cannot recognise that “JS”, “JavaScript” and “ECMAScript” refer to the same skill.

Distributed word representations such as Word2Vec [2] placed related words close together in a vector space. The Transformer architecture [3] and the models built on it, such as BERT [4], produced contextual representations in which the same word receives different vectors in different sentences. Sentence-BERT [5] made it practical to compute one embedding for a whole document or sentence and compare two of them by cosine similarity, which is the standard technique for semantic resume–job matching.

Large Language Models such as GPT-3 [6] and GPT-4 [7] go further: through a well-written prompt they can read a resume, extract structured fields, compare it with a job description, and justify the result in natural language, without any task-specific training. Retrieval-Augmented Generation [8] showed that answers become more reliable when the model is given retrieved source documents, an idea used in the proposed HR chatbot that answers only from the candidate database.

3.2 Reliability, Fairness and Compliance

LLMs are known to produce fluent but unsupported statements, a problem surveyed under the name hallucination [9]. For resume processing this means that a model could invent a degree, an employer or a skill. In addition, studies of hiring algorithms [10], [11] report that automated tools can reproduce or amplify existing biases and that vendors’ fairness claims are often hard to verify. A widely reported example is an experimental recruiting tool that was withdrawn after it was found to disadvantage women [12]. Regulators commonly use the four-fifths (80%) rule [13] to flag possible adverse impact when the selection rate of one group is less than four-fifths of that of the highest group.

3.3 Summary of the Reviewed Work

Reference	Work / Contribution	Relevance and limitation
[1] Salton & Buckley, 1988	TF-IDF term weighting for automatic text retrieval.	Basis of keyword-based similarity used in early ATS tools; matches only exact words, so synonyms and context are missed.
[2] Mikolov et al., 2013	Word2Vec: dense vector representations in which similar words lie close together.	Captures word similarity, but each word has a single context-free vector.
[3] Vaswani et al., 2017	Transformer architecture based on self-attention.	Foundation of modern NLP models and LLMs.

Reference	Work / Contribution	Relevance and limitation
[4] Devlin et al., 2019	BERT: deep bidirectional contextual language representations.	Large gains on language understanding tasks; needs fine-tuning for each task.
[5] Reimers & Gurevych, 2019	Sentence-BERT: sentence embeddings compared efficiently with cosine similarity.	Widely used for semantic matching of a resume against a job description.
[6] Brown et al., 2020; [7] OpenAI, 2023	GPT-3 and GPT-4: LLMs that perform tasks from prompts without task-specific training.	Enable parsing, scoring and explanation through prompting; outputs can be wrong or invented.
[8] Lewis et al., 2020	Retrieval-Augmented Generation: answers grounded in retrieved documents.	Motivates answering HR questions from the candidate database instead of from model memory.
[9] Ji et al., 2023	Survey of hallucination in natural language generation.	Shows why generated or extracted facts must be verified against a source.
[10] Raghavan et al., 2020; [11] Bogen & Rieke, 2018	Studies of bias, transparency and equity in algorithmic hiring tools.	Highlight the need for explainability, auditing and human oversight.
[12] Dastin, 2018; [13] EEOC, 1978	Reported withdrawal of a biased automated recruiting tool; the four-fifths rule for detecting adverse impact in selection rates.	Basis for the proposed blind screening and selection-rate fairness audit.

Table 3.1: Summary of the literature reviewed

In addition to the research literature, the features of commonly available ATS and AI-recruitment platforms were studied: resume parsing, job matching, candidate ranking, interview scheduling and recruitment analytics. These tools are largely recruiter-centred and treat the scoring model as a black box. The survey shows that the technology for semantic matching exists; what is missing is an integrated, explainable and verifiable system that serves both recruiters and candidates.

4. Existing Gaps

From the survey of the literature and of existing tools, the following gaps were identified. Each gap is paired with the way the proposed system addresses it.

Existing gap	Effect	Proposed solution in HireSense AI
Keyword-only matching	Good candidates are rejected because of different wording; irrelevant resumes pass by keyword stuffing.	Semantic, context-aware matching with separate weight for required and optional skills.

Existing gap	Effect	Proposed solution in HireSense AI
Black-box scores	Recruiters and candidates cannot see why a score was given.	Explainable report: sub-scores, matched and missing skills, reasons and an AI recommendation.
Recruiter-only focus	Candidates get no feedback on how to improve.	Candidate portal with ATS report, improvement suggestions, skill gap analysis and career assistant.
Unverified AI extraction	LLMs may invent skills, employers or contact details.	Every extracted item is checked against the resume text and unsupported items are removed.
Bias and fairness	Automated screening can disadvantage groups; no built-in audit.	Blind screening mode, selection-rate (four-fifths) audit and human-in-the-loop decisions.
Fragmented tools	Screening, comparison, scheduling and analytics are done in separate tools.	One platform covering the whole recruitment workflow.
Duplicates and versions	The same resume is processed repeatedly; candidates cannot compare versions.	Hash-based duplicate detection and resume version comparison.
No learning from decisions	Recruiters' corrections are lost.	Recruiter feedback is compared with the AI and used to suggest a shortlist threshold.
No operational visibility	Cost, errors and user actions are not monitored.	API usage/error monitoring and audit logs for administrators.

Table 4.1: Existing gaps and how the proposed work addresses them

5. Objectives of the proposed work

5.1 Main Objective

To develop an intelligent, explainable and secure web-based resume screening and recruitment assistant that reduces the effort of screening resumes while keeping the final decision with the recruiter.

5.2 Specific Objectives

1. To extract structured information (contact details, education, experience, projects, certifications, technical and soft skills) from PDF and text resumes and to store it in a relational database.
2. To match each resume with a job description semantically, considering required and optional skills, experience relevance and education, and to produce a ranked list with a score from 0 to 100.

3. To make every score explainable by showing sub-scores, matched and missing skills and the reasons behind the recommendation.
4. To validate all AI-extracted details against the original resume text so that invented information is detected and removed (hallucination control).
5. To support batch screening of many resumes for one job, including detection of duplicate resumes.
6. To help candidates through ATS compatibility reports, resume improvement suggestions, skill gap analysis, resume version comparison, job recommendations and a generative AI career assistant.
7. To support recruiters with candidate comparison, shortlisting and rejection, interview question generation, interview scheduling with invitation drafts and a chatbot that answers from the candidate database.
8. To reduce bias through blind screening and a selection-rate fairness audit, and to keep a human in the final decision.
9. To provide administrators with user and job management, skills demand analytics, shortlisting statistics, API usage and error monitoring and audit logs.
10. To learn from recruiter feedback by comparing human decisions with AI recommendations and suggesting an improved shortlisting threshold.

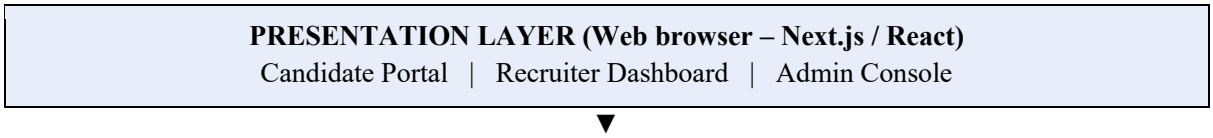
6. Proposed Methodologies

6.1 Development Methodology

The project follows an iterative (agile-style) development model. The system is divided into six modules (Table 6.2). Each module is designed, implemented and reviewed in a short cycle, starting with the core screening flow and then adding the candidate portal, interview tools, analytics and administration. Requirements are gathered from the study of existing ATS tools and the literature survey.

6.2 System Architecture

The system uses a layered architecture. The browser interface calls server-side API routes; the routes check the user's role, read and write the MySQL database and, where intelligence is needed, call the AI model through a single wrapper that records usage and errors. The OpenAI key and database credentials are kept only on the server.



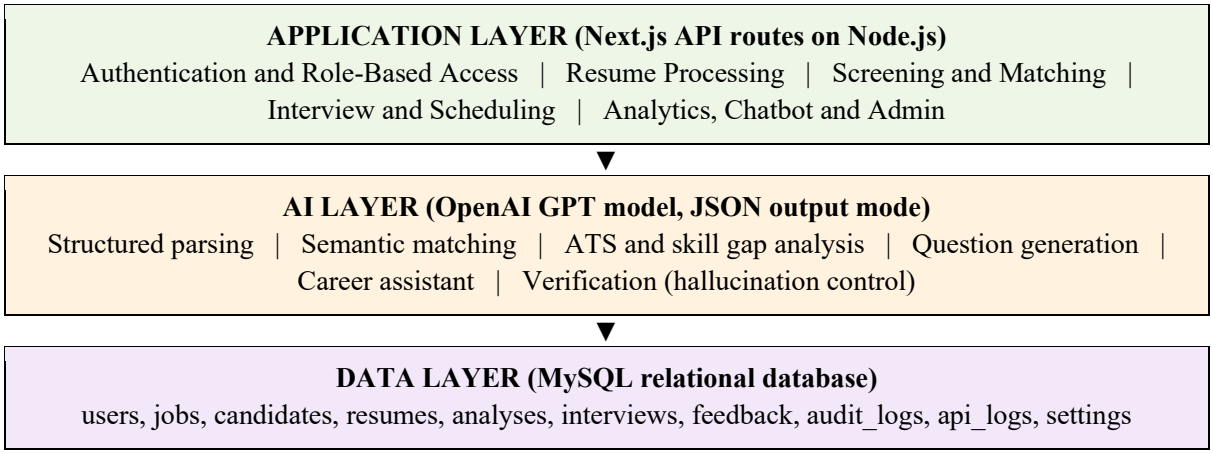


Figure 6.1: Layered architecture of HireSense AI

6.3 Technology Stack

Component	Technology	Purpose
Frontend	React 18 with Next.js 14 (App Router), plain CSS	User interface for the three roles; SVG charts (radar, donut, bars) without external chart libraries.
Backend	Node.js, Next.js API routes	REST-style endpoints, authentication, role checks, business logic.
Database	MySQL 8 (mysql2 driver, JSON columns)	Persistent storage of users, jobs, resumes, candidates, analyses and logs.
AI / NLP	OpenAI GPT model (gpt-4o-mini) with JSON response mode	Parsing, semantic matching, explanations, ATS analysis, skill gap, interview questions, chatbot.
Resume extraction	pdf-parse library; plain-text reader	Converts PDF and TXT resumes to text.
Security	scrypt password hashing, signed HttpOnly session cookie	Safe credential storage and session handling.
Tools	VS Code, Git/GitHub, MySQL Workbench, Postman	Development, version control, database design and API testing.

Table 6.1: Technology stack

6.4 Modules of the System

Module	Functions
1. Candidate Dashboard	Registration and login, profile management, resume upload (multiple versions), resume analysis history, ATS compatibility report, improvement suggestions, skill gap analysis, personalised job recommendations, application tracking.
2. Recruiter Dashboard	Recruiter authentication, company profile, create and publish jobs, required and optional skills, upload multiple resumes, automatic screening, candidate comparison, shortlist/reject, application status management.

Module	Functions
3. AI Resume Processing	Extraction of name and contact details, education, work experience, projects and certifications; identification of technical and soft skills; detection of resume sections; conversion of unstructured text into structured JSON.
4. Intelligent AI Matching	Resume versus job description comparison, semantic skill matching, experience relevance, education matching, matching percentage, missing-skill detection, ranking, explainable matching report.
5. Generative AI Career Assistant	Rewriting weak bullet points, improving summaries, suggesting keywords, generating project descriptions, suggesting skills to learn, resume question-answering chatbot, tailoring suggestions to a specific job.
6. Admin and Analytics	Managing candidates, recruiters and job postings; total resume analysis; application statistics; skills demand analytics; shortlisting statistics; API usage and error monitoring; audit logs.

Table 6.2: Modules and their functions

6.5 Resume Processing and AI Pipeline

1. **Upload:** the user uploads one or many PDF or text resumes.
2. **Text extraction:** the pdf-parse library (or a plain-text reader) converts each file to text; empty or unreadable files are rejected with a friendly message.
3. **Duplicate check:** a SHA-256 hash of the normalised text is compared with stored hashes so that identical resumes are not processed twice.
4. **AI analysis:** the resume text and the job description (including required and optional skills) are sent to the language model, which must reply in a fixed JSON format containing the candidate profile, match scores, skill overlap, reasons and a recommendation.
5. **Verification:** every extracted education, employer, project, certification, skill, e-mail and phone number is checked against the original text; unsupported items are dropped.
6. **Storage and ranking:** the normalised result is stored in MySQL and candidates are ranked by score.
7. **Human decision:** the recruiter reviews the explainable report and shortlists, keeps for review or rejects the candidate.

6.6 Matching and Scoring

The model evaluates four aspects – skills, experience, domain and education – and returns an overall score from 0 to 100 together with a per-skill match percentage for every required and optional skill of the job. Required skills are given higher importance than optional skills. For the job recommendation shown to candidates, a transparent formula is used:

$$Match\ \% = 100 \times (2 \times R_{found} + O_{found}) / (2 \times R_{total} + O_{total})$$

where R is the number of required skills and O the number of optional skills of the job, and "found" means the skill appears in the resume. The recruiter-side score uses the language model so that equivalent skills (for example "Express" and "Node.js backend") are recognised, and the reasons are shown to the user for verification.

6.7 Hallucination Control

Because language models can state facts that are not in the document, the system never trusts the extracted details blindly. After parsing, each extracted item is searched in the original resume text. Items that cannot be found are removed, and the profile screen reports how many items were confirmed and how many were removed. Contact details are kept only if they literally appear in the resume.

6.8 Fairness and Human Oversight

- **Blind screening:** a recruiter can switch on a mode in which names, e-mail addresses, phone numbers and file names are not sent from the server, so candidates appear as "Candidate #id".
- **Fairness audit:** shortlisting rates are compared across groups (for example experience bands and cities) and the impact ratio is checked against the four-fifths rule; a score-consistency check flags overall scores that do not match their sub-scores.
- **Human in the loop:** the AI only recommends. The final status (Shortlisted, Review, Rejected) is always set by a person. Protected personal characteristics are not collected or used for scoring.

6.9 Feedback Learning

Every status change made by a recruiter is stored together with the AI recommendation. The administrator can see the percentage of decisions in which the recruiter agreed with the AI and a suggested shortlisting threshold derived from the scores of candidates the recruiters actually shortlisted. The threshold can be applied with one click and then governs the automatic labelling of new screenings.

6.10 Database Design

Table	Purpose	Main fields
users	Accounts of all roles	id, name, email, password_hash, role, company
jobs	Job openings	title, description, required_skills, optional_skills, published

Table	Purpose	Main fields
candidates	One row per screened or applied resume	job_id, score, sub-scores, skill_overlap, parsed, status, resume_hash
resumes	Candidate-uploaded resume versions	user_id, label, text, hash, parsed
analyses	History of ATS / improvement / skill-gap results	user_id, resume_id, kind, score, data
interviews	Scheduled interviews	candidate_id, scheduled_at, duration, mode
feedback	Recruiter decision vs AI recommendation	candidate_id, ai_recommendation, final_status, score
audit_logs / api_logs	Audit trail and AI usage/error monitoring	action, detail / name, ok, ms, tokens, error
settings	System settings	shortlist threshold

Table 6.3: Main database tables

6.11 Security and Access Control

- Passwords are stored as salted scrypt hashes; sessions use a signed, HttpOnly cookie.
- Role-based access control: every API route checks whether the logged-in user is a candidate, recruiter or administrator.
- The OpenAI API key and database password are stored in a server-side environment file and are never sent to the browser.
- An audit log records important actions (login, job changes, screening, status changes, applications).

6.12 Testing Plan

Area	Test case	Expected result
Authentication	Wrong password, duplicate email, access to a page of another role.	Friendly error; access denied.
Resume upload	Unsupported file type, empty or scanned PDF, very short text.	Clear message; no record created.
Duplicate detection	Same resume uploaded twice for one job.	Second copy is skipped.
Parsing and validation	Resume where the model returns an item not present in the text.	Item removed; verification count shown.
Matching	Strong, partial and weak resumes for one job.	Scores ordered consistently with expectation.
Interview scheduling	Two interviews at overlapping times.	Second booking rejected.
Blind screening	Toggle on and open candidate lists.	Names and contact details are not shown.

Area	Test case	Expected result
Admin monitoring	Force an AI error.	Error appears in API monitor and audit log.

Table 6.4: Planned test cases

6.13 Hardware and Software Requirements

- **Hardware:** a computer with at least 4 GB RAM (8 GB recommended), a dual-core processor and an internet connection for the AI service.
- **Software:** Windows/Linux/macOS, Node.js 18 or later, MySQL 8, a modern web browser, VS Code, Git, and an OpenAI API key.

7. Expected Outcome of the proposed work

7.1 Deliverables

- A working multi-role web application (candidate, recruiter, administrator) with the six modules described above.
- A MySQL database with sample data and an automatic schema set-up.
- Explainable matching reports, ATS reports, skill gap analyses and interview material generated for each candidate.
- Project report, user manual, test report and source code repository.

7.2 Expected Benefits

- **For recruiters:** much shorter first-round screening time, a consistent ranking with reasons, and all hiring activities in one place.
- **For candidates:** understanding of why a resume scores as it does, concrete suggestions to improve it, and visibility of the application status.
- **For the organisation:** reduced risk through verified extraction, bias checks, audit logs and cost/error monitoring.

7.3 Evaluation Plan

The system will be evaluated on a set of real or realistic resumes for a few job roles. Planned measures are:

- Agreement between the system's ranking and a manual ranking by human reviewers (for example rank correlation and precision of the top-k shortlist).
- Field-level accuracy of extracted details (name, e-mail, skills, education) and the number of unsupported items removed by the verification step.
- Average processing time per resume compared with manual screening.

- Error rate and response time from the API monitor, and a short usability survey with students acting as recruiters and candidates.

These measures will be reported in the final project report; the figures are targets to be verified and are not claimed in this synopsis.

8. Future scope of the work

- **Embedding-based search:** store sentence embeddings and use vector similarity to rank very large resume pools quickly and at lower cost.
- **More file formats:** support DOCX resumes and OCR for scanned PDFs.
- **Integrations:** automatic e-mail delivery of invitations, calendar synchronisation and integration with job portals and existing ATS software.
- **Video interview assistance:** live or recorded interviews with speech-to-text and AI-assisted evaluation under recruiter control.
- **Multilingual support:** resumes and job descriptions in several languages.
- **Scalability:** background job queues for large batches, caching, containerised deployment and cloud hosting.
- **Richer fairness tools:** more detailed bias metrics and explanations, and model fine-tuning using the accumulated recruiter feedback.
- **Mobile application:** a companion app for candidates to receive notifications and manage applications.

9. References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[2] T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.

[3] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[4] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[6] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[7] OpenAI, “GPT-4 technical report,” arXiv:2303.08774, 2023.

[8] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[9] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.

[10] M. Raghavan, S. Barocas, J. Kleinberg and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in Proc. ACM FAT* Conference, 2020, pp. 469–481.

[11] M. Bogen and A. Rieke, “Help wanted: An examination of hiring algorithms, equity, and bias,” Upturn, 2018.

[12] J. Dastin, “Amazon scraps secret AI recruiting tool that showed bias against women,” Reuters, Oct. 2018.

[13] Equal Employment Opportunity Commission et al., “Uniform Guidelines on Employee Selection Procedures,” 29 C.F.R. Part 1607, 1978.

[14] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft), Stanford University.

[15] Next.js Documentation, Vercel Inc. [Online]. Available: https://nextjs.org/docs

[16] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/

[17] OpenAI API Documentation. [Online]. Available: https://platform.openai.com/docs